

Embedded Systems Software

Embedded systems software is **specialized code** designed to **control machines, devices, or specific hardware** components, often operating under strict **real-time, power, and memory constraints**. Unlike general-purpose software, it is **dedicated to specific functions**, such as running a microwave, managing an automotive braking system, or controlling a drone.

Key Aspects of Embedded Software

- **Hardware-Centric:** Software is tightly coupled with hardware (microcontrollers/microprocessors), requiring direct manipulation of registers and memory mapping.
- **Performance Constraints:** Optimized for speed, reliability, and low power consumption rather than user-facing interface features.
- **Real-time & Reliable:** Often requires deterministic behaviour (tasks must finish within specific time constraints) and high stability.
- **Development Tools:** Commonly uses C, C++, or assembly language, developed within IDEs such as Keil uVision, STM32CubeIDE, and Arduino.

Types of Embedded System Software

Embedded software is often classified by its complexity and functional role. **Most common** of them are **Bare-metal, and RTOS**. Below we discussed some other embedded software along with these two.

- **Firmware (Bare-metal):** Code written directly onto hardware, such as non-volatile memory (ROM, Flash), often without an operating system, handling direct hardware control (e.g., in a TV remote).
- **Real-Time Operating Systems (RTOS):** Specialized OSs designed for time-critical, deterministic tasks, prioritizing responsiveness (e.g., FreeRTOS, VxWorks).
- **Embedded Applications/Software:** High-level code performing specific functions within the device (e.g., user interface management).
- **Middleware:** Software residing between the RTOS and applications, providing high-level functions like communication stacks or database management.
- **Device Drivers:** Software enabling the OS to communicate with specific hardware components like sensors or actuators.

Embedded Software Applications

Embedded software is prevalent across diverse industries, generally categorized by their operation scale. Some of them are:

- **Consumer Electronics:** Smartphones, digital cameras, washing machines, microwaves, and wearable devices.
- **Automotive Systems:** Engine control units (ECUs), ABS braking, airbags, and navigation systems.
- **Industrial Automation:** Programmable Logic Controllers (PLCs), robotics, and automated manufacturing systems.
- **Medical Devices:** Pacemakers, insulin pumps, and imaging systems.
- **Aerospace and Défense:** Flight control systems, radar systems, and missiles.
- **Communication Systems:** Routers, modems, and satellites.

Embedded Software Development Tools

- **Programming Languages:** C, C++, Embedded C, Python, and Java are commonly used for development.
- **Hardware Platforms:** Microcontrollers (MCUs) such as ARM Cortex-M, PIC, AVR, and DSPs, or microprocessor-based systems.

Embedded Development Platforms

Embedded development platforms are **hardware systems used to design, test, and implement embedded applications.**

Types of Platforms

a) Microcontroller-Based Platforms

- Used for real-time, low-level control
- Programmed using Embedded C

Examples:

- STM32
- Arduino
- ESP32
- PIC Microcontroller
- MSP430

b) Single-Board Computers (SBCs)

- Run operating systems
- Used for complex applications

Example:

- Raspberry Pi

Comparison Insight

Feature	Microcontroller	SBC
OS	No	Yes
Speed	Moderate	High
Power	Low	Higher
Use	Real-time control	Complex computing

STM32

STM32 is a family of **32-bit microcontrollers** developed by STMicroelectronics.

Key Features

- Based on **ARM Cortex-M processors**
- High performance and low power consumption
- Rich peripherals (GPIO, ADC, UART, SPI, I2C, Timers, etc.)
- Used in **industrial, robotics, IoT, and embedded systems**

Why Use STM32?

- Gives **low-level hardware control**
- Suitable for **professional and research-level embedded systems**
- Supports development tools like STM32CubeIDE

In short: **STM32 = Powerful, flexible, and industry-oriented embedded platform**

Arduino

Arduino is an **open-source electronics platform** designed for **beginners and rapid prototyping**.

Key Features

- Easy-to-use hardware and software
- Programming based on simplified C/C++
- Built-in functions like `digitalWrite()`, `pinMode()`
- Large community support

Why Use Arduino?

- Very **beginner-friendly**
- Quick to build prototypes
- No need to understand low-level hardware initially

In short: **Arduino = Simple, beginner-friendly, rapid prototyping platform**

ESP32

- Developed by Espressif Systems
- A powerful microcontroller with **built-in WiFi and Bluetooth**

Features

- Dual-core processor
- Ideal for **IoT applications**
- Supports Arduino IDE and ESP-IDF

Best for: **Smart devices, IoT, Wireless projects**

Raspberry Pi

- Developed by Raspberry Pi Foundation
- A **single-board computer (SBC)**, not just a microcontroller

Features

- Runs Linux OS
- Supports Python, C, and more
- Has GPIO pins for hardware control

Best for: **Full computing + embedded + AI + education**

PIC Microcontroller

- Developed by Microchip Technology
- One of the **earliest and widely used** microcontroller families

Features

- Available in 8-bit, 16-bit, 32-bit
- Used in **industrial and consumer electronics**

Best for: **Traditional embedded systems and control applications**

MSP430

- Developed by Texas Instruments
- Known for **ultra-low power consumption**

Features

- Ideal for **battery-powered devices**
- Simple architecture

Best for: **Wearables, Medical devices, Low-power systems**

Comparative Summary

Platform	Type	Best Use Case
STM32	Microcontroller	Advanced embedded systems
Arduino	Microcontroller board	Learning & prototyping
ESP32	Microcontroller	IoT & wireless systems
Raspberry Pi	Single-board computer	Full computing + embedded
PIC	Microcontroller	Industrial embedded systems
MSP430	Microcontroller	Low-power applications

Embedded C Programming

- Embedded C is a programming language that is **used in the development of Embedded Systems**.
- Embedded Systems are specialized systems designed to perform very specific functions or tasks. **Embedded System** is the **combination of**

hardware and software, and the **software** is generally known as **firmware** which is embedded into the system hardware.

- **Embedded C** is used to **program a wide range of microcontrollers and microprocessors**.

Some features of embedded C are presented below.

- Embedded C has some **additional data types and keywords**. There are some special datatypes in Embedded C like **sbit, sfr** which are used for addressing special function registers in memory.
- Embedded C allows us to **work with hardware** devices like **sensors, and input-output devices**.
- Embedded C **requires a smaller number of resources** to execute in comparison with high-level languages such as Python or Java.
- There are various **Embedded C compilers** to compile the embedded C program such as **Keil Compiler, SPJ Compiler, Embedded GNU C Compiler, etc.**
- Embedded Systems can be **classified into small-scale, medium-scale, and sophisticated** embedded systems.
- The devices like air conditioners, printers, and mobile phones that we use in our daily lives are programmed by embedded C.

Difference between C and Embedded C

Parameter	C	Embedded C
Definition	C is a general-purpose programming language that can be used to write any type of programs.	Embedded C is an extension of C programming language to write programs for embedded systems.
Hardware Dependency	The executable file generated by C compiler is hardware independent as long as the OS is same.	Different embedded systems provide different embedded C compiler with their own libraries, and their generated code can only be run in the same hardware.
OS Dependency	C programs generally run on top of an OS .	Embedded C programs don't require traditional OS . They run on embedded systems hardware with minimal kernels or RTOS.
Applications	It is used in wide variety of software applications, games, etc.	It is used to develop firmware for embedded systems, IoT devices, etc.

Key Characteristics of Embedded C

- **Efficiency:** In Embedded C, we can create an **efficient code** to **optimize the limited resources** available in embedded systems. It aims to **minimize memory usage** and **maximize performance**.
- **Direct Hardware Interaction:** Embedded C allows programmers to interact directly with hardware components, such as microcontrollers, sensors, actuators, and other peripherals. This direct interaction facilitates precise control over the hardware, critical in embedded applications.
- **Low-level Programming:** Embedded C involves low-level programming, which deals with hardware-specific details like memory addresses, I/O ports, and register manipulation. This level of control is essential for efficiently managing hardware resources.
- **Real-time Operations:** Embedded systems often operate in real-time environments, requiring precise timing and response to events. Embedded C allows programmers to handle real-time tasks efficiently.

Structure of Embedded C Program

- **Comments:** Comments are readable text written to help user understand the code easily. They are ignored by compiler and do not take up any memory in the final code. There are two types of comments, Single line comments and Multiline comments.
- **Preprocessor Directive:** In Embedded C Preprocessor Directives are represented using #include or #define. Preprocessor Directives are used to indicate a header file specific to a microprocessor or microcontroller which contains all the functions, SFR's and the bits in those SFR's. reg51 header file is used in case of 8051 microcontroller.
- **Global Variables:** Global variables as the name suggests are global to program that is they can be **accessed anywhere in the program**. Global variables are static variables and are placed in RAM memory locations.
- **Local Variables:** Local variables in contrast to global variables are **confined to their respective functions**. Normally these variables are placed in stack or registers. It is only valid within the function in which it is declared.
- **Function:** Function is a **group of statements** that **together performs a task**. A **function declaration** tells the compiler about the name, return type and parameter of the function. A **function definition** provides actual body of the function.
- **Main Function:** Every Embedded C program has one main function and may contain one or more functions in the main functions. The program execution starts from the main function, and it is a core of every execution.

If more than one main function is written in the code, then compiler will confuse from where to start the program execution.

Sample Program

```
#include <reg51.h> // Preprocessor Directive (Header file for 8051)

unsigned char count = 0; // Global Variable

void delay(void); // Function Declaration

void main(void) // Main Function
{
    while(1) // Infinite loop (typical in embedded systems)
    {
        P1 = count; // Send value to Port 1
        delay(); // Call delay function
        count++; // Increment global variable
    }
}

void delay(void) // Function Definition
{
    int i, j; // Local Variables
    for(i = 0; i < 100; i++)
    {
        for(j = 0; j < 1000; j++);
    }
}
```

Brief Explanation of the Program

Comments

- Used to explain the code (e.g., // Preprocessor Directive)
- Ignored by the compiler

Preprocessor Directive

- #include <reg51.h>
- Includes the header file for the 8051 Microcontroller
- Provides access to special function registers like P1

Global Variable

- unsigned char count = 0;
- Accessible throughout the program
- Used to store and update a value sent to Port 1

Function Declaration

- void delay(void);
- Informs the compiler about the function before it is used

Main Function

- Execution starts from main()
- Infinite loop (while(1)) ensures continuous operation
- Sends count to Port 1 and increments it

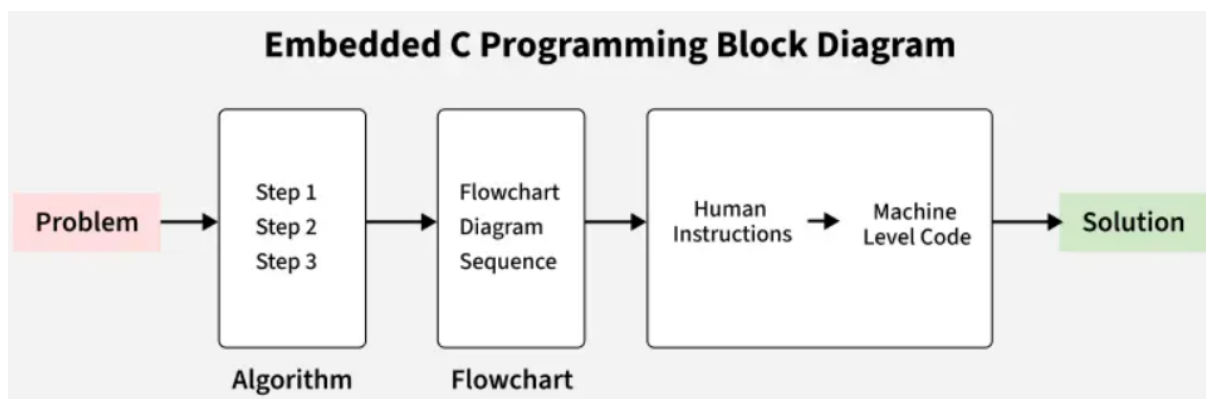
Function Definition

- delay() creates a time delay using nested loops
- Uses **local variables (i, j)** which exist only within the function

Standard Embedded C Data Types

Data Type	Bits	Range
Unsigned char	8	0 - 255
Signed char	8	-128 - +127
Unsigned int	16	0 - 65535
Signed int	16	-32768 - +32767
bit	1	0 - 1
sbit	1	0 - 1
sfr	8	0 - 255
sfr16	16	0 - 65535

Block Diagram Explanation of Embedded C



Problem

- Problem refers to a **challenge or task** which needs to be addressed by programming.
- The problem defines the **purpose and functionality** of the Embedded System.

- The necessary **coding solutions should be achieved by understanding the problem statement.**
- It **includes** functional requirements, non-functional requirements, Hardware Interfacing and Expected Output.

Algorithm

- Algorithm is a **set of instructions** that tells us that **how the task should be executed** in step-wise manner in order to solve the problem.
- Algorithm helps us to **visualize and design the logic** before implementation of actual program.
- It is **written in step-by-step order.**

Let's take a look at an **example** for better understanding. *Algorithm to add two numbers and store the result. The algorithm is:*

- Make the Port 1 and Port 2 as Input port.
- Take the data from Port 1.
- Take the data from Port 2.
- Add the content of Port 1 with Port 2
- Send result of addition to Port 3.

Flowchart

- A flowchart is a **graphical representation of an algorithm** which shows the steps to be followed **using some symbols and arrows.**
- Flowchart makes it easy to **understand the structure of program** before coding by **visualizing the flow of logic.**
- It **shows decision points** such as conditional statement, loops and code terminations.
- As it uses symbols and shapes, **debugging is easy** and it **takes less efforts in writing** the logic of a program.
- Flowchart and Algorithm **help in identifying logical errors** while troubleshooting the code which is a difficult process in itself.

Compilation and Uploading of Code

- Generally, **programmers write** code using **high level languages** like C where **human readable syntax** is used.
- This language is **not understandable by CPU** of computer. We **need to translate it into machine level code** which a CPU can easily understand.

- So, a **compiler needs** to be used for this purpose. Compiler is a specialized software tool which **translates the human readable code (source code) into machine code (binary code)** according to the specific microcontroller architecture.
- The **machine code is executed by the CPU** by carrying out the instructions step-by-step to perform the tasks written in the actual code written by programmer/user.
- Machine code consists of instructions which are understandable by the computer's CPU. It is a string of 0's and 1's representing logical, arithmetic operations.
- A **hex file is generated after compilation process** that contains the machine code which is **uploaded in microcontroller's flash memory** through programmer/debugger.
- After uploading, the **code is tested and verified** for correct operation of Embedded System.

In this way the **solution for the problem statement is achieved in Embedded C.**

Basic Embedded C Programming Steps

- **Requirement Analysis:** Understanding the requirements of the Embedded System to be developed according to the problem | requirement should be done.
- **Selecting Environment Setup:** Selection of proper tools such as choosing Integrated Development Environment (IDE), compiler, debugger and other necessary tools for Embedded C Programming.
- **Code Development:** Writing the Embedded C code based on the system requirements and design specifications must be done in this step. The program should consume less memory space, must be reliable and scalable.
- **Compilation Process:** In this stage the compiler translates the embedded C code into assembly language code or machine level code. The machine level code is in the form of 0's and 1's. Also, the preprocessor handles the directives such as #include, #define.
- **Loading to Target Device:** Uploading the compiled code onto the target hardware (microcontroller, FPGA) using tools like debugger or flash programmers needs to be done.
- **Execution and Debugging:** Run the embedded system and execution of code is performed. Employing debugging tools to identify and resolve any errors or issues in the code.

- **Documentation and Maintenance:** Creating proper documentation detailing the system architecture, code functionalities and memory usage. Periodically updating and maintaining the codebase to address issues.

Advanced Techniques for Embedded C

- **Pointer Manipulation:** Pointers in C are **powerful but can be complex**. They allow direct access to memory locations, aiding in efficient data manipulation. Understanding pointer arithmetic, dynamic memory allocation (malloc/free), and using pointers accessing hardware registers or structures are crucial in embedded C programming.
- **Interrupt Handling:** In embedded systems, interrupts are used to handle asynchronous events. Mastering ISR involves understanding how to write interrupt service routines, handle interrupt priorities, manage shared resources, and minimize interrupt latency to ensure timely response to events.
- **RTOS (Real-Time Operating Systems):** RTOS facilitates multitasking within embedded systems. Understanding concepts like task scheduling, context switching, inter-process communication (IPC), and synchronization mechanisms (semaphores, mutexes) is very much important for developing real-time embedded applications.
- **Peripheral Interfacing:** Embedded systems interact with various peripherals. Knowledge of communication protocols (UART, SPI, I2C), handling GPIO pins, configuring timers, and managing interrupts related to peripherals is important for effective interfacing.
- **Low-power Optimization:** Embedded devices often run on limited power. Techniques to reduce power consumption involve utilizing low-power modes provided by microcontrollers, selectively shutting down unused peripherals, and optimizing algorithms for energy efficiency.
- **Memory Management:** Embedded systems have limited memory. Efficiently managing memory includes minimizing memory fragmentation, choosing appropriate data types, implementing memory pooling, and handling dynamic memory allocation carefully to avoid memory leaks.
- **Debugging and Testing:** Embedded system debugging involves using hardware debuggers, emulators, simulators, and printf-style debugging.

Embedded C Program Examples

1. Write a Program to read the number 1 from port 1, number 2 from port 2, then add them, store the result, send it to Port 3.

```

#include<reg.51.h>
void main()
{
unsigned char a,b,c ;
P1 = 0XFF;    //make port 1 as input port
P2 = 0XFF;    //make port 2 as input port
a=P1;
b=P2;
c= a+b ;
P3= c;
}

```

2. Write a program to Turn on and off the LED with some delay.

```

#include<reg51.h>
sbit LED=P1.1;
void delay(void);
void main(void)
{
while(1)
{
LED=1;
delay();
LED=0;
delay();
}
}

void delay(void)
{
unsigned char i,k;
for(i=0;i<70;i++)
for(k=0;k<255;k++);
}

```

3. Write a program to transfer the data from port P0 to port P1.

```

#include<reg51.h>
void main (void )
{
unsigned char X;
P0=0XFF;    // P0 as input port
P1=0X00;    // P1 as output port

```

```
while(1)
{
X = P0;  // read port0
P1 = X;  // output data to port1
}
```

Advantages of Writing a Program in Embedded C

- It is **easy and less time consuming** to write code in Embedded C instead of assembly programming language.
- Embedded C **program is easier to modify and update**.
- Code available in function libraries can be used by the programmer.
- Embedded C code is **portable** to other microcontrollers with little or no modifications.
- Embedded C code tends to be **more readable and maintainable** than assembly language.

Disadvantages of Embedded C Programming

- It can **only perform one task at a time**; it cannot perform several activities.
- We **need to update the hardware** if we **changed the application**.
- Only the hardware system is supported.
- It is **not scalable**, scalability is a problem with Embedded C.
- It has limitations such as **restricted RAM**, which affects the computer's compatibility.

General Purpose Input/Output (GPIO)

General Purpose Input/Output (GPIO) is the **most fundamental peripheral** in an embedded system. It serves as the **primary bridge** between the **microcontroller (MCU)** and the **external physical world**, allowing the software to control or sense electrical signals.

Physical Structure and Pin Multiplexing

Most modern **microcontrollers** have **limited pins**, so each pin is "**multiplexed**." This means a **single physical pin** can be **configured to act** as a **GPIO**, an **ADC input**, a **UART transmit line**, or a **PWM output**.

- **Internal Structure:** A GPIO pin typically consists of a pair of complementary MOSFETs (one to pull the line to VCC, one to pull it to Ground) and an input buffer to read the state.
- **Protection:** Most pins include ESD (Electrostatic Discharge) protection diodes to prevent damage from static or voltage spikes.

Core Modes of Operation

A GPIO pin is **configured by writing to specific hardware registers**. There are **two primary modes**. They are discussed below.

(i) Output Mode

In this mode, the **MCU controls the voltage level** of the pin.

- **Push-Pull:** The most common output type. The pin is actively driven to either Logic High (VCC) or Logic Low (Ground).
- **Open-Drain:** The pin can only pull the line to Ground (Logic Low). To achieve a High state, an external **pull-up resistor** is required. This is commonly used in communication buses like I2C to allow multiple devices to share a single line.

(ii) Input Mode

In this mode, the **MCU "listens" to the voltage level** applied to the pin by an external circuit.

- **High Impedance (Floating):** The pin is in a neutral state. Without a clear signal, the pin may "float" between high and low due to electrical noise, leading to unpredictable data.
- **Pull-Up/Pull-Down:** To prevent floating, internal resistors can be enabled to gently "tie" the pin to VCC (Pull-Up) or Ground (Pull-Down) when no external signal is present.

Key Hardware Registers

Software interacts with GPIO through a set of memory-mapped registers. While names vary by manufacturer (STM32, AVR, PIC), the functions are universal. They are presented below along with small descriptions.

- **Direction Register (DDR/MODER):** Determines if the pin is an Input or an Output.

- **Data Output Register (ODR/PORT):** Sets the pin high or low when in output mode.
- **Data Input Register (IDR/PIN):** Reads the current logic level of the pin.
- **Pull-up/Pull-down Register:** Configures internal resistors.

Common GPIO Challenges

(i) Switch Debouncing

Mechanical buttons do not create a clean signal. When pressed, the metal contacts "bounce," creating a rapid series of high/low transitions for several milliseconds.

Solution: Software must implement a delay or a counter to ensure the signal has stabilized before registering a press.

(ii) Current Limits

GPIO pins have strict current limits (often between 5mA and 25mA).

The Rule: Never drive high-power components like motors or large relays directly from a GPIO pin. Use a transistor or an optoisolator as a driver.

(iii) Interrupt-on-Change

Instead of the CPU constantly checking (polling) a pin's state, most GPIOs can be configured to trigger an **Interrupt**. The CPU can sleep and only wake up when the pin detects a "Rising Edge" (low to high) or a "Falling Edge" (high to low).

Practical Example

Now, let's look at a clear and practical example for understanding GPIO. Here, we'll present **two programs**, one using **STM32 (HAL library)** and also briefly mention **Arduino** for comparison.

Example 1: GPIO Initialization (STM32 – HAL Library)

Scenario: Configure a GPIO pin (e.g., **PA5**) as an output and blink an LED.

Embedded C Code (STM32)

```
#include "stm32f4xx_hal.h" // Preprocessor Directive

// Function Prototypes
void SystemClock_Config(void);
```

```

static void GPIO_Init(void);

int main(void)
{
    HAL_Init();           // Initialize HAL Library
    SystemClock_Config(); // Configure system clock
    GPIO_Init();          // Initialize GPIO

    while (1)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5); // Toggle PA5
        HAL_Delay(500);                          // 500 ms delay
    }
}

// GPIO Initialization Function
static void GPIO_Init(void)
{
    __HAL_RCC_GPIOA_CLK_ENABLE(); // Enable clock for GPIO A

    GPIO_InitTypeDef GPIO_InitStructure = {0};

    GPIO_InitStructure.Pin = GPIO_PIN_5; // Select Pin PA5
    GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP; // Output Push-Pull
mode
    GPIO_InitStructure.Pull = GPIO_NOPULL; // No pull-up or pull-down
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_LOW;

    HAL_GPIO_Init(GPIOA, &GPIO_InitStructure); // Initialize GPIO
}

```

Explanation

1. Header File: stm32f4xx_hal.h provides access to STM32 HAL functions and register definitions.

2. HAL Initialization: HAL_Init() initializes system-level features like SysTick.

3. Clock Configuration

- SystemClock_Config() sets up the system clock (required for timing functions).

4. GPIO Initialization

- __HAL_RCC_GPIOA_CLK_ENABLE() → Enables clock for GPIO Port A.
- GPIO_InitTypeDef → Structure used to configure GPIO settings:

- Pin: Selects PA5
- Mode: Output mode
- Pull: No pull-up/down resistor
- Speed: Low speed

5. Main Loop

- HAL_GPIO_TogglePin() toggles the LED
- HAL_Delay(500) creates a delay → LED blinks every 500 ms

Example 2: GPIO (Arduino – Simpler Abstraction)

Code

```
void setup()
{
  pinMode(13, OUTPUT); // Set pin 13 as output
}

void loop()
{
  digitalWrite(13, HIGH); // Turn LED ON
  delay(500);

  digitalWrite(13, LOW); // Turn LED OFF
  delay(500);
}
```

Explanation

- pinMode(13, OUTPUT) → Configures GPIO pin
- digitalWrite() → Sets HIGH/LOW
- delay() → Creates delay

Arduino hides low-level details (like registers and clocks), while **STM32 gives fine-grained hardware control**.

STM32 vs Arduino

Feature	STM32 (HAL)	Arduino
Control Level	Low-level / Hardware control	High-level abstraction
Complexity	Higher	Very simple
Flexibility	Very high	Limited
Learning Focus	Embedded systems design	Rapid prototyping

Real-Time Operating System (RTOS)

- A real-time operating system (RTOS) is a **specialized operating system** designed to **handle time-sensitive tasks** with precise timing constraints, ensuring predictability and stability.
- These systems are **crucial in** applications like industrial automation, robotics, medical devices, and embedded systems where **delays or failures can have severe consequences**.
- Real-time operating systems are also commonly **used in high-stakes environments (e.g., aerospace and defense)** where real-time responses are essential for safety and performance.

The Evolution of Real-time Operating Systems

- **1950s–1960s: The Foundations:** While real-time concepts were experimented with in the late 1940s, formal RTOS development began in the 1960s to manage time-sensitive, safety-critical applications. Early systems aimed to improve on simple batch processing by handling external events promptly.
- **1960s–1970s: The First Systems:** IBM developed the "Basic Executive" (1962), often considered an early RTOS, featuring interrupt handling and I/O support, followed by Executive II in 1963. These systems introduced preemption and better scheduling in military and industrial sectors.
- **1980s–1990s: Commercialization & Standardisation:** RTOS became commercialized with products like VxWorks and QNX, widely adopted for telecommunications and embedded systems. POSIX standards were introduced, helping to unify the design and portability of RTOS.
- **2000s–Present: IoT and Advanced Embedded Systems:** The growth of embedded, Internet of Things (IoT), and autonomous systems necessitated lightweight, efficient RTOS like FreeRTOS. Modern RTOS, such as those used in industrial automation, ensure strict reliability while integrating IoT connectivity and AI.

Key Technological Shifts

- **Scheduling Evolution:** Shifted from simplistic approaches to robust, deterministic techniques like Rate Monotonic Scheduling (RMS) for static tasks and Earliest Deadline First (EDF) for dynamic scheduling.
- **Resource Management:** Evolved from simple interrupt handling to advanced message-passing systems, semaphores (A semaphore is a synchronization tool in operating systems—an integer variable used to

manage concurrent access to shared resources by multiple processes or threads), and memory management.

- **Specialization vs. Integration:** RTOS evolved from highly specialized, standalone kernels to Real-time Linux variants or microkernels.

Difference Between an GPOS/OS and an RTOS

A general-purpose operating system (GPOS) and a real-time operating system (RTOS) both **coordinate system hardware resources** (e.g., CPU, memory, I/O devices, storage), yet they **differ significantly** in their focus and capabilities.

Operating systems—such as Microsoft Windows, Linux and Unix—**focus on maximizing overall system efficiency and supporting multitasking**, but they **rely on non-deterministic scheduling**. As **non-real-time systems**, they may **not always complete tasks on time**, particularly under heavy load or in virtual machine (VM) environments where resources are shared.

In contrast to a general-purpose OS, a **real-time OS** is **designed for real-time applications** and **guarantees that tasks meet strict timing requirements**, often within microseconds. The resources in a real-time system are managed with **deterministic scheduling** to ensure that **high-priority tasks are completed within specific time frames**, even under load. While an RTOS can support VMs, the overhead of virtualization may impact its ability to meet real-time demands.



Fig. Real time embedded system with RTOS

A comparative table between RTOS and GPOS is presented below.

Characteristic	RTOS	General-Purpose OS
Primary Goal	Meeting deadlines with deterministic behaviour	Maximizing throughput and user experience
Response Time	Guaranteed, predictable response times	Best-effort, variable response times
Scheduling	Priority-based preemptive scheduling	Time-sharing, fair scheduling
Resource Usage	Minimal overhead, optimized for efficiency	Feature-rich, higher resource consumption
Complexity	Lightweight, focused functionality	Complex, supporting diverse applications
Cost	Higher development and licensing costs	Generally lower cost

Application Domain	Embedded systems, industrial control, aerospace	Desktop computers, servers, mobile devices
Examples	FreeRTOS, VxWorks, QNX, eCos	Windows, macOS, Linux, Unix

Key Features of a Real-time Operating System

Key features of a real-time operating system (RTOS) include the following.

- **Determinism:** An RTOS guarantees tasks are executed within a predictable, fixed time frame, which is essential for time-sensitive applications.
- **Multitasking:** Multitasking in a RTOS involves scheduling and managing multiple tasks, determining the order in which tasks are executed, and switching between them rapidly, giving the impression of parallel execution.
- **Memory Management:** An RTOS prevents tasks from interfering with each other's memory space, improving system stability and security.
- **Minimal Latency:** An RTOS reduces response time to external events or interrupts, ensuring quick reactions in real-time environments and minimizing interrupt latency. Efficient context switching in an RTOS further minimizes task-switching latency, allowing tasks to be quickly swapped in and out of the CPU, which reduces delays between executions and enhances overall system responsiveness.
- **Priority-based Scheduling:** An RTOS executes higher-priority tasks before lower-priority ones, making sure critical tasks are handled first.
- **Resource Allocation:** An RTOS efficiently handles memory allocation, processing power and other system resources to support real-time performance.
- **Interrupt Handling:** An RTOS quickly and efficiently responds to hardware or software interrupts using RTOS application programming interface (API) mechanisms. This capability minimizes the time spent handling interrupts and ensures real-time task completion.
- **Task Synchronization:** An RTOS provides inter-task communication (ITC) with mechanisms like semaphores and message queues to synchronize tasks and ensure safe sharing of resources among multiple tasks.

Types of Real-time Operating Systems

There are **three main types** of real-time operating systems (RTOSes), each designed to handle varying levels of timing precision (often in milliseconds) and tolerance for missed deadlines. These are:

- Hard real-time operating systems
- Firm real-time operating systems
- Soft real-time operating systems

Hard Real-time Operating Systems

- Hard real-time operating systems are designed with **strict timing constraints** for when meeting deadlines is crucial.
- Any **failure to meet a deadline** can have **severe consequences**, making reliability paramount.
- Common applications of hard real-time operating systems include **aerospace, robotics and industrial control systems**.
- **To meet these stringent requirements, file systems** in hard real-time systems are **often streamlined to reduce overhead**, ensuring that data is accessed or written within the strict timing constraints.

Firm Real-time Operating Systems

- Firm real-time operating systems **typically require meeting deadlines**, but they can **tolerate occasional delays** without causing significant issues.
- Examples of such systems include **multimedia playback, networking and certain industrial automation applications**.

Soft Real-time Operating Systems

- Soft real-time operating systems **focus on timely execution**, but **missing a deadline does not have critical consequences**.
- The system can **still function properly**, albeit with **reduced performance**.
- Examples include **desktop operating systems, web servers and some office automation tools**.

Benefits of an RTOS

A real-time operating system (RTOS) offers **numerous key benefits**, making it **ideal for critical, resource-constrained applications**. Some of these are:

- **Reliability and Predictability:** An RTOS operates consistently and reliably, even under heavy load. This minimizes the risk of failure, which is essential for critical applications.
- **Minimal Overhead:** Designed to be lightweight, an RTOS ensures efficiency in resource-constrained environments, helping maintain overall system performance.
- **Fault Tolerance:** An RTOS ensures that the system continues to operate reliably, even in the presence of errors or faults. This capability is crucial for mission-critical and safety-sensitive environments where uptime and stability are paramount.
- **Enhanced System Efficiency:** An RTOS optimizes resource utilization by ensuring that the highest priority tasks are completed first while using minimal overhead, leading to better performance in resource-constrained environments.
- **Improved Safety:** In safety-critical applications (e.g., medical devices, automotive systems, aerospace), a real-time operating system's deterministic behaviour and fault tolerance ensure that the system behaves predictably and remains operational, even under fault conditions, which directly improves safety.
- **Scalability:** An RTOS can efficiently manage both simple and complex systems, enabling scalability. Applications can be expanded or integrated with additional resources without compromising real-time performance.
- **System Stability:** By providing memory protection, task synchronization and error recovery mechanisms, a real-time OS helps to prevent system crashes, data corruption and unexpected behaviours, ensuring long-term system stability.

Common RTOS Use Cases

Real-time operating systems are **used in a variety of industries where precise timing, reliability and predictable behaviour are essential** for critical applications. Some of these are:

- **Aerospace:** Real-time operating systems are utilized in aerospace systems for flight control, navigation and mission-critical applications where timing precision is essential. Using high-performance computing (HPC), these critical systems can process complex data from various sensors in

real time, ensuring quick, accurate responses that are crucial for safety and performance.

- **Robotics:** In robotics, real-time operating systems ensure the real-time control of robotic movements, sensor processing and communication. These systems need to operate with high precision and low latency, especially in industrial automation, medical robots and autonomous vehicles.
- **Industrial Control Systems:** An RTOS is commonly applied in industrial control systems, such as manufacturing processes, plant automation and assembly lines. These systems require tight timing constraints to monitor sensors and other equipment in real time.
- **Healthcare Devices:** An RTOS is used in medical devices like pacemakers, infusion pumps and diagnostic equipment, where timely and predictable operation is necessary to ensure patient safety and device reliability.
- **Automotive Systems:** In automotive applications, real-time operating systems support critical functions—such as autonomous driving systems and advanced driver-assistance systems (ADAS)—where safety and responsiveness are key.
- **Telecommunications:** An RTOS is essential in telecommunications infrastructure, including mobile base stations and satellite communication systems, where real-time processing and low-latency performance are necessary for maintaining stable and fast connections.
- **Defense and Military:** An RTOS is used in defense and military applications for radar systems, weapons control and surveillance systems, where operational accuracy and speed are crucial to establishing mission success and safety.

Popular Real-time Operating Systems

These popular real-time operating systems (RTOSes) are **designed to meet the specific needs of various industries**. They are **optimized to run on processors like Intel and ARM**, ensuring high performance, reliability and efficiency across diverse applications. Some of these are:

- **VxWorks:** This highly reliable RTOS, used in aerospace, defense and industrial automation, is known for its scalability, security and safety features.
- **QNX:** This modular, POSIX-compliant RTOS is favoured in automotive, medical and industrial sectors, offering reliability and fault tolerance

- **FreeRTOS:** This open-source, lightweight RTOS is ideal for embedded systems, microcontrollers, IoT devices and consumer electronics, emphasizing simplicity and efficiency.
- **RTEMS:** This open-source RTOS is designed for high-performance embedded systems, commonly deployed in aerospace, telecommunications and robotics applications.
- **embOS:** This compact RTOS is known for its scalability and deterministic performance, and it is often used in industrial and medical systems where reliability is key.
- **Zephyr:** This open-source, scalable RTOS from the Linux Foundation is optimized for small, resource-constrained devices, such as IoT gadgets and wearables.
- **ThreadX:** This high-performance RTOS, known for its minimal footprint and efficient resource management, is frequently used in consumer electronics, automotive and industrial systems.

Why Use an RTOS instead of a "Bare-Metal" Loop?

Feature	Bare-Metal (Firmware)	RTOS
Complexity	Simple for small projects.	Scalable for complex systems.
Responsiveness	High-priority events might wait for the loop.	High-priority tasks preempt immediately.
Maintainability	Hard to add features without breaking timing.	Easy to add tasks; the scheduler manages timing.
Power Management	Manual implementation required.	Kernel can automatically enter "Idle" or "Sleep".